Documento de Definición y Validación de Dominio

TPI Backend 2026: Sistema Integral de Gestión de Turnos y Recursos (MediTurnos API)

1. Explicación del Dominio y Contexto del Problema

En el sector de la salud privada e independiente, la gestión eficiente de turnos médicos, recursos físicos (consultorios asignados) y equipamiento de diagnóstico representa un desafío. Tradicionalmente, la falta de coordinación genera superposición de turnos y alta inasistencia.

El dominio abarca la Gestión Integral de Reservas y Recursos. El sistema operará como una plataforma backend orientada a microservicios diseñada deliberadamente para mantener una baja complejidad operativa. Su objetivo central es coordinar la agenda médica, asignando automáticamente espacios físicos fijos por profesional y gestionando interacciones automatizadas, todo esto soportado por un diseño modular y desacoplado.

2. Propuesta de Valor y Actores Involucrados

- **Pacientes (ROL_PACIENTE):** Autogestión 24/7 para consultar disponibilidad, reservar, reprogramar o cancelar turnos, interactuando principalmente a través de notificaciones y un Chatbot de WhatsApp.
- **Profesionales de la Salud (ROL_PROFESIONAL):** Control unificado de agenda. Cada médico tiene asignado un consultorio fijo y distintos tipos de servicios (con sus respectivas duraciones y costos).
- **Administradores (ROL_ADMIN):** Gestión del catálogo de sedes, mantenimiento de consultorios/equipamientos y supervisión del estado general de las reservas.
- **(Gestión de identidades externa):** La administración de identidades, almacenamiento seguro de credenciales y roles se delega en su totalidad al Identity Provider Keycloak (OIDC + JWT), quitándole la carga de seguridad interna a la lógica de negocio del sistema.

3. Procesos y Reglas de Negocio Clave

El sistema no operará como un simple CRUD. Incorpora lógica de dominio a través de las siguientes reglas core:

1. **Disponibilidad Cruzada (Compatibilidad Tripartita):** Un turno solo puede reservarse si, para la franja horaria solicitada, existe disponibilidad del médico Y el servicio no colisiona con el equipamiento necesario. Al tener consultorios fijos

asignados por profesional, la validación se agiliza sin perder rigor en la asignación del espacio físico.

2. **Transacción Directa y Atómica:** Al momento de la reserva de un turno, la operatoria es directa. No se utilizan bloqueos temporales de celdas horarias ("Hold States") ni patrones asíncronos complejos como Sagas para este flujo. Si el recurso está disponible en el instante de la transacción, el turno se genera e impacta inmediatamente en base de datos.
3. **Regla de Cancelación y Reprogramación:** El paciente puede cancelar su cita hasta 24 horas antes sin penalidad. Si el profesional cancela un bloque de atención, el sistema dispara automáticamente un evento de alerta para ofrecer reprogramaciones prioritarias. (Se excluye explícitamente el manejo de validaciones con Obras Sociales para acotar el alcance).

4. Arquitectura y Matriz de Validación Técnica

Criterio Evaluado (TPI)	Implementación Arquitectónica Propuesta
División de Microservicios	Se implementarán estrictamente dos (2) microservicios funcionales (Spring Boot) para minimizar la carga operativa: 1. ms-catalogo-recursos: Responsable del ABM y catálogo de médicos, especialidades, tipos de servicios (duraciones y costos) y sedes físicas. 2. ms-reservas: Responsable de orquestar el ciclo de vida transaccional del turno y coordinar las notificaciones.
Diseño Interno de Capas	Dentro de cada microservicio se aplicará un estricto patrón de capas (Controller -> Service -> Repository). Los controladores solo mapean DTOs y gestionan HTTP, delegando toda la lógica de negocio (como las validaciones cruzadas) a la capa @Service, la cual interactúa con las bases de datos a través de abstracciones de Spring Data JPA (JpaRepository).
Colaboración entre Microservicios	Interacción Síncrona (RestClient): Cuando un paciente intenta agendar un turno en ms-reservas (iniciador del flujo), este servicio realiza una petición síncrona mediante RestClient a ms-catalogo-recursos. Solicita validar si el tipo de servicio existe, recuperar su duración y costo, y verificar si el médico está activo. Solo con la respuesta favorable (200 OK) de catálogo, ms-reservas persiste la transacción.
API Pública Externa	Se integrará la API de Twilio / WhatsApp Business (o SendGrid) para dar soporte al Chatbot de WhatsApp, permitiendo

	consultar disponibilidad y gestionar recordatorios automatizados.
Seguridad y Enrutamiento	Se utilizará Spring Cloud Gateway como punto único de entrada al backend. Este Gateway implementará un Security Filter que delegará la autenticación en Keycloak. Los microservicios solo aceptarán peticiones enrutadas desde el Gateway con tokens JWT válidos.
Componente Común (JAR Reutilizable)	Se desarrollará un artefacto Maven independiente (mediturnos-common-lib.jar). Contendrá DTOs compartidos (para la comunicación entre ms-reservas y ms-catalogo) y el manejo centralizado de excepciones (como GlobalExceptionHandler). Integración: Será instalado en el repositorio local .m2 mediante mvn clean install e inyectado como <dependency> en ambos microservicios.
Persistencia de Datos (Base de Datos)	Se utilizará una única instancia de base de datos compartida (H2 embebida para desarrollo rápido, escalable a PostgreSQL contenedorizado para el despliegue final). Para garantizar el desacoplamiento, se aplicará una división lógica mediante esquemas independientes (ej. schema_catalogo y schema_reservas), asegurando que ningún microservicio acceda directamente a las tablas del otro.
Documentación y Despliegue	Cada microservicio expondrá sus contratos dinámicamente mediante Swagger UI / OpenAPI 3. El despliegue integral (Gateway, microservicios, PostgreSQL y Keycloak) se orquestará localmente mediante Docker Compose.

5. Registro de Decisiones de Arquitectura (ADRs)

- **ADR 001: Adopción de "Única BD con División Lógica por Esquemas"**
  • Contexto: El patrón estricto de "Database-per-Service" requiere levantar múltiples motores de bases de datos, lo cual multiplica los costos de infraestructura, consumo de RAM y complejidad de despliegue, siendo excesivo para las necesidades del proyecto actual.
  • Decisión: Se levanta un único motor de base de datos relacional (PostgreSQL). Sin embargo, cada microservicio operará de forma aislada sobre su propio esquema lógico protegido. No habrá Joins ni consultas SQL directas entre dominios. Toda sincronización de información entre esquemas deberá hacerse sí o sí a través de las llamadas API (RestClient) entre los microservicios.

• Consecuencias: Se optimiza radicalmente el uso de recursos y se facilita el desarrollo local, pero se mantiene la cohesión lógica y se cumple con la restricción académica de bajo acoplamiento de datos exigida por la cátedra.

- **ADR 002: Modelo de Comunicación Síncrona vs Asíncrona**
  • Contexto: Al reservar un turno, ms-reservas necesita datos vitales (duración y costo) que residen en ms-catalogo-recursos.
  • Decisión: Se opta por una comunicación síncrona vía HTTP (RestClient) en lugar de replicar datos asíncronamente mediante colas de mensajería (RabbitMQ/Kafka).
  • Consecuencias: Simplifica enormemente la topología de la red y el manejo de transacciones, a expensas de un ligero aumento en la latencia al momento de agendar. Se implementarán tiempos de espera (timeouts) defensivos para evitar que una caída en catálogo bloquee el pool de conexiones de reservas.